

1. **Query:** Display all students.

SQL Script:

```
SELECT user_id, name, rating, college
FROM "User"
WHERE role = 'Student';
```

Output:

	user_id [PK] integer	name character varying (100)	email character varying (100)	password character varying (255)	role character varying (20)	rating integer	college character varying (100)	created_at timestamp without time zone
1	16	Hinal Rangnai	hinal.mnc@daict.ac.in	hashed_stud_16	Student	1850	DA-IICT	2025-02-10 12:00:00
2	17	Aarav Sharma	aarav.sharma@iitb.ac.in	hashed_stud_17	Student	2100	IIT Bombay	2025-02-10 13:00:00
3	18	Ananya Iyer	ananya.iyer@iitd.ac.in	hashed_stud_18	Student	1920	IIT Delhi	2025-02-10 14:00:00
4	19	Kabir Verma	kabir.verma@bits.ac.in	hashed_stud_19	Student	1650	BITS Pilani	2025-02-10 15:00:00
5	20	Diya Patel	diya.patel@nitw.ac.in	hashed_stud_20	Student	1580	NIT Warangal	2025-02-10 16:00:00
6	21	Rohan Das	rohan.das@nsut.ac.in	hashed_stud_21	Student	1740	NSUT Delhi	2025-02-11 09:00:00
7	22	Isha Gupta	isha.gupta@dtu.ac.in	hashed_stud_22	Student	1420	DTU Delhi	2025-02-11 10:00:00
8	23	Dev Patel	dev.patel@daict.ac.in	hashed_stud_23	Student	2210	DA-IICT	2025-02-11 11:00:00
9	24	Rahul Mishra	rahul.mishra@iitb.ac.in	hashed_stud_24	Student	1420	IIT Bombay	2025-02-11 12:00:00
10	25	Priyanka Reddy	priyanka.r@iith.ac.in	hashed_stud_25	Student	1710	IIT Hyderabad	2025-02-11 13:00:00
11	26	Amit Shah	amit.shah@daict.ac.in	hashed_stud_26	Student	1600	DA-IICT	2025-02-12 14:00:00
12	27	Bhavna Pandey	bhavna.p@iitd.ac.in	hashed_stud_27	Student	1820	IIT Delhi	2025-02-12 15:00:00
13	28	Chirag Paswan	chirag.p@bits.ac.in	hashed_stud_28	Student	1490	BITS Pilani	2025-02-12 16:00:00
14	29	Deepak Hooda	deepak.h@nitw.ac.in	hashed_stud_29	Student	1650	NIT Warangal	2025-02-12 17:00:00
15	30	Esha Deol	esha.deol@nsut.ac.in	hashed_stud_30	Student	2010	NSUT Delhi	2025-02-12 18:00:00
16	31	Gautam Gambhir	gautam.g@dtu.ac.in	hashed_stud_31	Student	1540	DTU Delhi	2025-02-13 10:00:00
17	32	Hardik Pandya	hardik.p@daict.ac.in	hashed_stud_32	Student	1680	DA-IICT	2025-02-13 11:00:00
18	33	Ishant Sharma	ishant.s@iitb.ac.in	hashed_stud_33	Student	1720	IIT Bombay	2025-02-13 12:00:00
19	34	Jasprit Bumrah	jasprit.b@iitd.ac.in	hashed_stud_34	Student	1390	IIT Delhi	2025-02-13 13:00:00
20	35	Krunal Pandya	krunal.p@bits.ac.in	hashed_stud_35	Student	1850	BITS Pilani	2025-02-13 14:00:00
21	36	Lokesh Rahul	lokesh.r@nitw.ac.in	hashed_stud_36	Student	1660	NIT Warangal	2025-02-14 15:00:00
22	37	Manish Pandey	manish.p@nsut.ac.in	hashed_stud_37	Student	1590	NSUT Delhi	2025-02-14 16:00:00
23	38	Navdeep Saini	navdeep.s@dtu.ac.in	hashed_stud_38	Student	1920	DTU Delhi	2025-02-14 17:00:00
24	39	Rishabh Pant	rishabh.p@daict.ac.in	hashed_stud_39	Student	1430	DA-IICT	2025-02-14 18:00:00
25	40	Shreyas Iyer	shreyas.i@iitb.ac.in	hashed_stud_40	Student	1780	IIT Bombay	2025-02-14 19:00:00
26	41	Yuzvendra Chahal	yuzi.c@iitd.ac.in	hashed_stud_41	Student	1500	IIT Delhi	2025-02-15 09:00:00
27	42	Bhuvneshwar Kumar	bhuvi.k@bits.ac.in	hashed_stud_42	Student	1630	BITS Pilani	2025-02-15 10:00:00
28	43	Mohammad Shami	shami.m@nitw.ac.in	hashed_stud_43	Student	1810	NIT Warangal	2025-02-15 11:00:00
Total rows: 85 Query complete 00:00:00.096 LF Ln 1, Col 8								

2. **Query:** Show all contests.

SQL Script:

```
SELECT contest_id, title, start_time, end_time
FROM "Contest";
```

Output:

	contest_id [PK] integer	title character varying (100)	start_time timestamp without time zone	end_time timestamp without time zone
1	1	CodeSprint Round 1	2026-03-10 18:00:00	2026-03-10 20:00:00
2	2	Data Structures Faceoff	2026-03-17 18:00:00	2026-03-17 21:00:00
3	3	Weekly Challenge #3	2026-03-24 19:00:00	2026-03-24 21:00:00
4	4	Dynamic Programming Special	2026-04-01 15:00:00	2026-04-01 18:00:00
5	5	Grand Coders League	2026-04-15 14:00:00	2026-04-15 17:00:00
6	6	Graph Theory Blitz	2026-05-01 19:00:00	2026-05-01 21:00:00
7	7	Summer Code Clash	2026-06-10 12:00:00	2026-06-10 15:00:00
8	8	ByteSize Cup 2026	2026-07-05 18:00:00	2026-07-05 19:30:00
9	9	Advanced Graph Theory Invitational	2026-07-20 15:00:00	2026-07-20 18:00:00
10	10	Independence Day Coding Championship	2026-08-15 14:00:00	2026-08-15 18:00:00

3. **Query:** List all Easy problems.

SQL Script:

```
SELECT problem_id, title
FROM "Problem"
WHERE difficulty = 'Easy';
```

Output:

	problem_id [PK] integer 	title character varying (100) 
1	1	Two Sum
2	2	Valid Parentheses
3	3	Merge Sorted Arrays
4	4	Palindrome Number
5	5	Reverse Linked List
6	6	FizzBuzz Ultimate
7	7	Binary Search Baseline
8	8	Single Number Locator
9	9	Valid Anagram
10	10	Maximum Subarray Sum
11	11	Climbing Stairs
12	12	Invert Binary Tree
13	13	Contains Duplicate
14	14	Length of Last Word
15	15	Pascal Triangle Generator
16	41	Valid Palindrome II
17	53	Fibonacci Sequence Base
18	54	Valid Palindrome Baseline

4. **Query:** Find students from DA-IICT.

SQL Script:

```
SELECT name, rating  
FROM "User"  
WHERE college = 'DA-IICT';
```

Output:

	name character varying (100) 	rating integer 
1	Manish Sharma	0
2	Rajesh Patel	0
3	Siddharth Desai	0
4	Hinal Rangnai	1850
5	Dev Patel	2210
6	Amit Shah	1600
7	Hardik Pandya	1680
8	Rishabh Pant	1430
9	Shardul Thakur	1580
10	Prithvi Shaw	1940
11	Ravichandran Ashwin	1750
12	Avesh Khan	1710
13	Shahbaz Ahmed	1400
14	Sai Sudharsan	1480
15	Rahul Tewatia	1870
16	Akash Madhwal	1830

5. **Query:** Show all accepted submissions.

SQL Script:

```
SELECT submission_id, user_id, problem_id
FROM "Submission"
WHERE verdict = 'Accepted';
```

Output:

	submission_id [PK] integer 	user_id integer 	problem_id integer 
1	1	16	1
2	2	17	1
3	4	18	1
4	5	16	2
5	7	20	1
6	8	21	1
7	9	22	1
8	10	23	1
9	11	24	3
10	12	25	1
11	13	26	1
12	14	27	1
13	15	28	2
14	16	29	2
15	17	30	2
16	18	31	3
17	19	32	3
18	20	33	3
19	21	34	41
20	22	35	41
21	23	36	41
22	24	37	41
23	25	38	1
24	26	39	1
25	27	40	2
26	29	16	17
27	30	23	17
28	31	53	28
29	32	16	30
30	34	17	19

6. **Query:** Display all available programming languages.

SQL Script:

```
SELECT *  
FROM "Language";
```

Output:

	language_id [PK] integer 	language_name character varying (50) 
1	1	C++17
2	2	Python 3.10
3	3	Java 17
4	4	Go 1.18
5	5	Rust 2021
6	6	C (GCC 11.2)
7	7	JavaScript (Node.js 16)
8	8	TypeScript (v4.7)
9	9	Ruby (v3.1)
10	10	PHP (v8.1)
11	11	Kotlin (v1.7)
12	12	Swift (v5.6)
13	13	Scala (v3.1)
14	14	Haskell (GHC 9.2)
15	15	C# (Mono 6.12)
16	16	C# (.NET 6)
17	17	Python 2.7 (Legacy)
18	18	C++20 (GCC 12.1)
19	19	Java 11 (LTS)
20	20	Ruby 3.2
21	21	Go 1.20
22	22	Rust 2024 Edition
23	23	Kotlin 1.9
24	24	TypeScript 5.0
25	25	C17 (GCC 12.2)
26	26	Perl 5.36

7. **Query:** Number of users from each college.

SQL Script:

```
SELECT college, COUNT(*) AS total_students
FROM "User"
GROUP BY college
ORDER BY total_students DESC;
```

Output:

	college character varying (100) 🔒	total_students bigint 🔒
1	DA-IICT	16
2	IIT Delhi	14
3	IIT Bombay	14
4	BITS Pilani	14
5	NIT Warangal	13
6	NSUT Delhi	13
7	DTU Delhi	12
8	IIT Hyderabad	2
9	IIT Roorkee	1
10	NIT Trichy	1

8. **Query:** Average rating of students.

SQL Script:

```
SELECT AVG(rating) AS average_rating
FROM "User"
WHERE role='Student';
```

Output:

	average_rating numeric 🔒
1	1664.0000000000000000

9. **Query:** Top 10 highest-rated students.

SQL Script:

```
SELECT name, rating
FROM "User"
WHERE role='Student'
ORDER BY rating DESC
LIMIT 10;
```

Output:

	name character varying (100) 🔒	rating integer 🔒
1	Dev Patel	2210
2	Aarav Sharma	2100
3	Esha Deol	2010
4	Shivam Mavi	1950
5	Prithvi Shaw	1940
6	Ananya Iyer	1920
7	Navdeep Saini	1920
8	Tilak Varma	1910
9	Mohammed Siraj	1890
10	Yash Dayal	1890

10. **Query:** Number of problems in each difficulty.

SQL Script:

```
SELECT difficulty, COUNT(*) AS total
FROM "Problem"
GROUP BY difficulty;
```

Output:

	difficulty character varying (10) 🔒	total bigint 🔒
1	Easy	18
2	Medium	24
3	Hard	12

11. **Query:** Number of submissions for each verdict.

SQL Script:

```
SELECT verdict, COUNT(*) AS total
FROM "Submission"
GROUP BY verdict;
```

Output:

	verdict character varying (30) 🔒	total bigint 🔒
1	Accepted	30
2	Wrong Answer	2
3	Runtime Error	2

12. **Query:** Students who registered for Contest 1.

SQL Script:

```
SELECT u.name
FROM "User" u
JOIN "Contest_Registration" cr
ON u.user_id = cr.user_id
WHERE cr.contest_id = 1;
```

Output:

	name character varying (100) 🔒
1	Hinal Rangnai
2	Aarav Sharma
3	Ananya Iyer
4	Kabir Verma
5	Diya Patel
6	Rohan Das
7	Rahul Mishra
8	Shreyas Iyer

13. **Query:** Problems with their tags.

SQL Script:

```
SELECT p.title, t.tag_name
FROM "Problem" p
JOIN "Problem_Tag" pt
ON p.problem_id = pt.problem_id
JOIN "Tag" t
ON pt.tag_id = t.tag_id;
```

Output:

	title character varying (100) 	tag_name character varying (50) 
1	Two Sum	Arrays
2	Valid Parentheses	Stacks
3	Merge Sorted Arrays	Arrays
4	Palindrome Number	Arrays
5	Reverse Linked List	Linked Lists
6	FizzBuzz Ultimate	Math & Sequences
7	Binary Search Baseline	Binary Search
8	Single Number Locator	Bit Manipulation
9	Valid Anagram	Strings
10	Maximum Subarray Sum	Arrays
11	Climbing Stairs	Dynamic Programming
12	Invert Binary Tree	Trees
13	Knapsack 0/1 Problem	Dynamic Programming
14	Longest Common Subsequence	Dynamic Programming
15	XOR Permutations	Bit Manipulation
16	3Sum Zero Tracker	Two Pointers
17	Container With Most Water	Two Pointers
18	Longest Substring Without Repeating	Sliding Window
19	Group Anagrams Together	Strings
20	Product of Array Except Self	Arrays
21	Coin Change Optimization	Dynamic Programming
22	Course Schedule DFS	Graphs
23	Pacific Atlantic Water Flow	Graphs
24	Palindromic Substrings Count	Strings
25	Edit Distance Engine	Dynamic Programming
26	Segment Tree Bounds Update	Arrays
27	Merge K Sorted Lists	Linked Lists
28	Trapping Rain Water	Arrays
29	Valid Palindrome II	Strings
30	Kth Largest Element	Arrays
31	Network Delay Time	Graphs
32	Binary Tree Maximum Path Sum	Trees

14. **Query:** Number of problems in each contest.

SQL Script:

```
SELECT contest_id,  
COUNT(problem_id) AS total_problems  
FROM "Contest_Problem"  
GROUP BY contest_id;
```

Output:

	contest_id integer	total_problems bigint
1	3	5
2	5	5
3	4	6
4	6	4
5	2	4
6	7	4
7	1	4

15. **Query:** Top 5 students according to leaderboard rank.

SQL Script:

```
SELECT u.name, l.rank  
FROM "Leaderboard" l  
JOIN "User" u  
ON l.user_id=u.user_id  
ORDER BY l.rank  
LIMIT 5;
```

Output:

	name character varying (100)	rank integer
1	Isha Gupta	1
2	Priyanka Reddy	1
3	Kabir Verma	1
4	Hinal Rangnai	1
5	Chirag Paswan	1

16. **Query:** Student with maximum rating.

SQL Script:

```
SELECT name,rating
FROM "User"
WHERE rating=(
SELECT MAX(rating)
FROM "User"
WHERE role='Student'
);
```

Output:

	name character varying (100) 🔒	rating integer 🔒
1	Dev Patel	2210

17. **Query:** Contest winners.

SQL Script:

```
SELECT c.title,
u.name,
l.rank
FROM "Leaderboard" l
JOIN "Contest" c
ON l.contest_id=c.contest_id
JOIN "User" u
ON l.user_id=u.user_id
WHERE l.rank=1;
```

Output:

	title character varying (100) 🔒	name character varying (100) 🔒	rank integer 🔒
1	CodeSprint Round 1	Hinal Rangnai	1
2	Data Structures Faceoff	Kabir Verma	1
3	Weekly Challenge #3	Isha Gupta	1
4	Dynamic Programming Special	Priyanka Reddy	1
5	Grand Coders League	Chirag Paswan	1
6	Graph Theory Blitz	Gautam Gambhir	1
7	Summer Code Clash	Jasprit Bumrah	1
8	ByteSize Cup 2026	Manish Pandey	1
9	Advanced Graph Theory Invitational	Shreyas Iyer	1
10	Independence Day Coding Championship	Mohammed Shami	1

18. **Query:** Number of accepted submissions by each student.

SQL Script:

```
SELECT u.name,  
COUNT(s.submission_id) AS accepted  
FROM "User" u  
JOIN "Submission" s  
ON u.user_id=s.user_id  
WHERE s.verdict='Accepted'  
GROUP BY u.name  
ORDER BY accepted DESC;
```

Output:

	name character varying (100) 🔒	accepted bigint 🔒
1	Hinal Rangnai	4
2	Dev Patel	2
3	Aarav Sharma	2
4	Bhavna Pandey	1
5	Chirag Paswan	1
6	Deepak Hooda	1
7	Diya Patel	1
8	Esha Deol	1
9	Gautam Gambhir	1
10	Hardik Pandya	1
11	Isha Gupta	1
12	Ishant Sharma	1
13	Jasprit Bumrah	1
14	Krunal Pandya	1
15	Lokesh Rahul	1
16	Manish Pandey	1
17	Navdeep Saini	1
18	Prithvi Shaw	1
19	Priyanka Reddy	1
20	Rahul Mishra	1
21	Rishabh Pant	1
22	Rohan Das	1
23	Shreyas Iyer	1
24	Amit Shah	1
25	Ananya Iyer	1

19. **Query:** Student who solved the most problems.

SQL Script:

```
SELECT u.name,  
COUNT(DISTINCT s.problem_id) AS solved  
FROM "User" u  
JOIN "Submission" s  
ON u.user_id=s.user_id  
WHERE s.verdict='Accepted'  
GROUP BY u.name  
ORDER BY solved DESC  
LIMIT 1;
```

Output:

	name character varying (100) 🔒	solved bigint 🔒
1	Hinal Rangnai	4

20. **Query:** Contest having maximum participants.

SQL Script:

```
SELECT c.title,  
COUNT(cr.user_id) AS participants  
FROM "Contest" c  
JOIN "Contest_Registration" cr  
ON c.contest_id=cr.contest_id  
GROUP BY c.title  
ORDER BY participants DESC  
LIMIT 1;
```

Output:

	title character varying (100) 🔒	participants bigint 🔒
1	CodeSprint Round 1	8

21. **Query:** Highest-rated student from each college.

SQL Script:

```
SELECT college,  
MAX(rating) AS highest_rating  
FROM "User"  
WHERE role='Student'  
GROUP BY college;
```

Output:

	college character varying (100) 🔒	highest_rating integer 🔒
1	IIT Hyderabad	1710
2	IIT Bombay	2100
3	DTU Delhi	1920
4	NIT Warangal	1910
5	NSUT Delhi	2010
6	BITS Pilani	1890
7	DA-IICT	2210
8	IIT Delhi	1920

22. **Query:** Students who participated in both Contest 1 and Contest 2.

SQL Script:

```
SELECT u.name
FROM "User" u
JOIN "Contest_Registration" c1
ON u.user_id=c1.user_id
JOIN "Contest_Registration" c2
ON u.user_id=c2.user_id
WHERE c1.contest_id=1
AND c2.contest_id=2;
```

Output:

	name character varying (100) 🔒
1	Hinal Rangnai
2	Rahul Mishra
3	Shreyas Iyer

23. **Query:** Problems never submitted.

SQL Script:

```
SELECT title
FROM "Problem"
WHERE problem_id NOT IN
(
SELECT DISTINCT problem_id
```

FROM "Submission"

);

Output:

	title character varying (100) 
1	Palindrome Number
2	Reverse Linked List
3	FizzBuzz Ultimate
4	Binary Search Baseline
5	Single Number Locator
6	Valid Anagram
7	Maximum Subarray Sum
8	Climbing Stairs
9	Invert Binary Tree
10	Contains Duplicate
11	Length of Last Word
12	Pascal Triangle Generator
13	Knapsack 0/1 Problem
14	XOR Permutations
15	Container With Most Water
16	Group Anagrams Together
17	Product of Array Except Self
18	Coin Change Optimization
19	Course Schedule DFS
20	Pacific Atlantic Water Flow
21	Palindromic Substrings Count
22	Validate Binary Search Tree
23	Top K Frequent Elements
24	House Robber Stashes
25	Edit Distance Engine
26	Segment Tree Bounds Update
27	Median of Two Sorted Arrays
28	Regular Expression Matching
29	Merge K Sorted Lists
30	Trapping Rain Water
31	Sliding Window Maximum
32	N-Queens Placement Matrix
33	Kth Largest Element
34	Network Delay Time
35	Binary Tree Maximum Path S...
36	Subarray Sum Equals K
37	Word Search Grid
38	Longest Palindromic Substring
39	Binary Tree Zigzag Traversal
40	K Closest Points to Origin
41	Minimum Window Substring
42	Word Ladder Transformation
43	Remove Invalid Parentheses
44	Fibonacci Sequence Base
45	Valid Palindrome Baseline

24. **Query:** Get the full leaderboard for a contest sorted by rank.

SQL Script:

```
SELECT l.rank, u.name, l.solved, l.penalty
FROM "Leaderboard" l
JOIN "User" u
ON l.user_id = u.user_id
WHERE l.contest_id = 1
ORDER BY l.rank;
```

Output:

	rank integer	name character varying (100)	solved integer	penalty integer
1	1	Hinal Rangnai	2	57
2	2	Aarav Sharma	1	14
3	3	Ananya Iyer	1	64

25. **Query:** Show a user's submission history with per-test-case results .

SQL Script:

```
SELECT s.submission_id, p.title, s.verdict, sr.testcase_id, sr.status,
sr.runtime
FROM "Submission" s
JOIN "Problem" p
ON s.problem_id = p.problem_id
JOIN "Submission_Result" sr
ON s.submission_id = sr.submission_id
WHERE s.user_id = 16
ORDER BY s.submitted_at;
```

Output:

	submission_id integer	title character varying (100)	verdict character varying (30)	testcase_id integer	status character varying (30)	runtime double precision
1	1	Two Sum	Accepted	2	AC	0.01
2	1	Two Sum	Accepted	1	AC	0.005
3	5	Valid Parentheses	Accepted	3	AC	0.022

26. **Query:** List all problems in a contest with their tags and points.

SQL Script:

```
SELECT cp.problem_code, p.title, cp.points, t.tag_name
FROM "Contest_Problem" cp
JOIN "Problem" p
ON cp.problem_id = p.problem_id
JOIN "Problem_Tag" pt
ON p.problem_id = pt.problem_id
```

```

JOIN "Tag" t
ON pt.tag_id = t.tag_id
WHERE cp.contest_id = 1
ORDER BY cp.problem_code;

```

Output:

	problem_code character (1)	title character varying (100)	points integer	tag_name character varying (50)
1	A	Two Sum	100	Arrays
2	B	Valid Parentheses	250	Stacks
3	C	Merge Sorted Arrays	300	Arrays
4	D	Valid Palindrome II	400	Strings

27. **Query:** Find all badges a user has earned and when they were awarded .

SQL Script:

```

SELECT b.badge_name, ub.awarded_at
FROM "User_Badge" ub
JOIN "Badge" b
ON ub.badge_id = b.badge_id
WHERE ub.user_id = 16;

```

Output:

	badge_name character varying (100)	awarded_at timestamp without time zone
1	First AC Achievement	2026-03-10 18:12:00
2	Dynamic Prodigy	2026-03-10 21:00:00
3	Contest Champion	2026-06-10 12:15:00
4	Fast Tracker	2026-03-10 18:12:00
5	Bug Hunter	2026-03-10 18:24:00
6	Tree Climber	2026-06-10 14:12:00

28. **Query:** Display a user's rating history over all contests.

SQL Script:

```

SELECT c.title, rh.old_rating, rh.new_rating, rh.rating_change,
rh.updated_at
FROM "Rating_History" rh
JOIN "Contest" c
ON rh.contest_id = c.contest_id
WHERE rh.user_id = 16
ORDER BY rh.updated_at;

```

Output:

	title character varying (100) 🔒	old_rating integer 🔒	new_rating integer 🔒	rating_change integer 🔒	updated_at timestamp without time zone 🔒
1	CodeSprint Round 1	1800	1850	50	2026-03-10 21:00:00
2	Data Structures Faceoff	1850	1885	35	2026-03-17 22:00:00
3	Weekly Challenge #3	1885	1910	25	2026-03-24 22:00:00
4	Dynamic Programming Special	1885	1920	35	2026-04-01 19:00:00
5	Summer Code Clash	1850	1895	45	2026-06-10 16:00:00

29. **Query:** Find Problems a User Has Not Yet Solved in an Active Contest.

SQL Script:

```
SELECT p.problem_id, p.title
FROM "Contest_Problem" cp
JOIN "Problem" p
ON cp.problem_id = p.problem_id
WHERE cp.contest_id = 1
AND p.problem_id NOT IN
(
    SELECT problem_id
    FROM "Submission"
    WHERE user_id = 16
    AND verdict = 'Accepted'
);
```

Output:

	problem_id [PK] integer 🔒	title character varying (100) 🔒
1	3	Merge Sorted Arrays
2	41	Valid Palindrome II

30. **Query:** Retrieve the Editorial for a Problem After Its Release Time.

SQL Script:

```
SELECT p.title, e.content, e.release_time
FROM "Editorial" e
JOIN "Problem" p
ON e.problem_id = p.problem_id
WHERE p.problem_id = 1
AND CURRENT_TIMESTAMP >= e.release_time;
```

Output:

	title character varying (100) 🔒	content text 🔒	release_time timestamp without time zone 🔒
1	Two Sum	Use a linear scan tracking map array indices inverse complements.	2026-03-10 20:15:00

Triggers:

1. Update Total Submissions for a Problem

--This trigger automatically increases the total submission count

--whenever a new submission is made for a problem.

SQL Script:

```
CREATE OR REPLACE FUNCTION update_total_submissions()
RETURNS TRIGGER AS $$
BEGIN
    UPDATE "Problem_Statistics"
    SET total_submissions = total_submissions + 1
    WHERE problem_id = NEW.problem_id;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER total_submission_trigger
AFTER INSERT ON "Submission"
FOR EACH ROW
EXECUTE FUNCTION update_total_submissions();
```

2. Update Accepted Submission Count

--This trigger automatically increases the accepted submission count

--when a submission gets Accepted.

SQL Script:

```
CREATE OR REPLACE FUNCTION update_accepted_submissions()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.verdict = 'Accepted' THEN
        UPDATE "Problem_Statistics"
        SET accepted_submissions = accepted_submissions + 1
        WHERE problem_id = NEW.problem_id;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER accepted_submission_trigger
```

```
AFTER INSERT ON "Submission"
FOR EACH ROW
EXECUTE FUNCTION update_accepted_submissions();
```

3. Notify User After Accepted Submission

--This trigger inserts a notification whenever a user's submission is Accepted.

SQL Script:

```
CREATE OR REPLACE FUNCTION notify_user_after_accept()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.verdict = 'Accepted' THEN
        INSERT INTO "Notification"
        (user_id, message, notification_type, created_at, is_read)
        VALUES
        (
            NEW.user_id,
            'Congratulations! Your solution has been Accepted.',
            'Submission',
            CURRENT_TIMESTAMP,
            FALSE
        );
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER accepted_notification_trigger
AFTER INSERT ON "Submission"
FOR EACH ROW
EXECUTE FUNCTION notify_user_after_accept()
```